

CORS (Cross-Origin Resource Sharing) - Spring Boot Notes

1. Same-Origin Policy (SOP)

Same-Origin Policy is a browser security mechanism that blocks requests between different origins by default.

An origin consists of:

- Protocol (http/https)
- Domain
- Port

Example:

Frontend: `http://localhost:5173`

Backend: `http://localhost:8080`

Different ports mean different origins, so the browser blocks communication unless CORS is enabled.

2. What is CORS?

CORS allows a server to explicitly permit trusted frontend applications to access its APIs.

The server sends HTTP headers such as:

Access-Control-Allow-Origin

Access-Control-Allow-Methods

Access-Control-Allow-Headers

Access-Control-Allow-Credentials

3. CORS Request Flow

React Frontend

|

| API Request

v

Browser checks CORS

|

v

Spring Boot Backend

|

| Sends CORS headers

v

Browser allows response

4. CORS Headers

Request Headers:

Origin:

Shows the frontend URL making the request.

Access-Control-Request-Method:

Specifies the HTTP method that will be used.

Access-Control-Request-Headers:

Shows custom headers that will be sent.

Response Headers:

Access-Control-Allow-Origin:

Defines allowed frontend origins.

Access-Control-Allow-Methods:

Defines allowed HTTP methods.

Access-Control-Allow-Headers:

Defines allowed request headers.

Access-Control-Allow-Credentials:

Allows cookies or authentication credentials.

5. Simple vs Preflight Requests

Simple Request:

The browser directly sends the request.

Example:

GET /api/students

Preflight Request:

The browser first sends an OPTIONS request.

Flow:

Browser

|

| OPTIONS

v

Server

|

| Permission granted

v

Browser sends actual request

6. Spring Boot CORS Configuration Methods

Method 1: @CrossOrigin Annotation

Example:

```
@CrossOrigin(  
    origins = "http://localhost:5173",  
    methods = {RequestMethod.GET, RequestMethod.POST},  
    allowedHeaders = "*" )
```

Used for:

- Small projects
- Quick fixes
- Individual controllers

Method 2: WebMvcConfigurer

Global configuration:

```
@Configuration  
public class CorsConfig implements WebMvcConfigurer {  
  
    public void addCorsMappings(CorsRegistry registry){  
        registry.addMapping("/**")  
            .allowedOrigins("http://localhost:5173");  
    }  
}
```

Used for:

- Medium and large applications
- Centralized CORS management

Method 3: CorsFilter

A filter-based approach for advanced control.

Method 4: Spring Security cors()

When Spring Security is used:

```
http.cors(Customizer.withDefaults());
```

Security configuration must enable CORS.

7. Common CORS Errors and Fixes

Error:

No 'Access-Control-Allow-Origin' header is present.

Reason:

Server is not sending CORS headers.

Fix:

Add `@CrossOrigin` or global CORS configuration.

Error:

Response to preflight has invalid HTTP status code.

Reason:

OPTIONS request is blocked, commonly by Spring Security.

Fix:

Allow OPTIONS requests.

Error:

Wildcard '*' not allowed when credentials are true.

Reason:

Credentials cannot be combined with all origins.

Wrong:

```
allowedOrigins("*")
```

```
allowCredentials(true)
```

Correct:

```
allowedOrigins("http://localhost:5173")
```

```
allowCredentials(true)
```

Error:

Works in Postman but fails in browser.

Reason:

Postman does not enforce CORS. Browsers do.

Fix:

Configure backend CORS correctly.

8. Important Interview Points

Q: Why use CORS?

Answer:

CORS allows trusted frontend applications running on different origins to access backend APIs securely.

Q: Why does Postman work but browser fails?

Answer:

Postman does not enforce Same-Origin Policy, while browsers enforce CORS for security.

Q: What is a preflight request?

Answer:

A browser-generated OPTIONS request sent before certain cross-origin requests to check whether the actual request is allowed.

9. Final Summary

- Same-Origin Policy blocks cross-origin calls by default.
- CORS uses HTTP headers to allow trusted origins.
- Preflight requests use OPTIONS method.
- Spring Boot supports:

1. @CrossOrigin

2. WebMvcConfigurer

3. CorsFilter

4. Spring Security cors()

- For JWT/Spring Security applications, configure CORS inside Security configuration.